ISA 401: Business Intelligence & Data Visualization

04: Data Import & Export

Fadel M. Megahed, PhD

Raymond E. Glos Professor in Business
Farmer School of Business
Miami University

@Fadel Megahed
fmegahed
fmegahed@miamioh.edu
Automated Scheduler for Office Hours

Fall 2026

Before We Start

1 Join today's Zoom

Use the Zoom link in the **Week 0 Module** on Canvas. **Mute** your mic and **turn off** your video.
Log in from the **lab's computer** so you can **share your screen** if you need help with an error.

2 Get your repository ready

Open **GitHub Desktop**. If `business_intelligence` is listed, click **Fetch origin**.
Not on this computer? **File → Clone repository** into **M:\isa401**, exactly like last week.

3 Open the project

In the repo folder, double-click **business_intelligence.Rproj** (or RStudio → **File → Open Project**).
Check the top right corner of RStudio before you type anything: it must say `business_intelligence`.

4 Start today's markdown

File → New File → R Markdown..., title *Class 04*, **OK**. Delete everything below the setup chunk, then save as **markdowns/class04.Rmd**.
Exactly like Monday's demo, this time on your own.

Done early? **Help the person next to you.** Stuck? **Share your screen on Zoom** and we fix it together.

Warm-Up: Monday in Three Questions



Learning Objectives for Today's Class

- **Create and inspect data frames** and **lists**; run `dplyr::glimpse()` on anything you did not build yourself.
- **Subset and index** those structures with `[ ]`, `[[ ]]`, and `$`.
- **Import** data from **CSV files**, **Excel workbooks**, and **web-served JSON files**.
- **Export** results with `write_csv()` and `write_xlsx()`, then **knit and push** the lot.

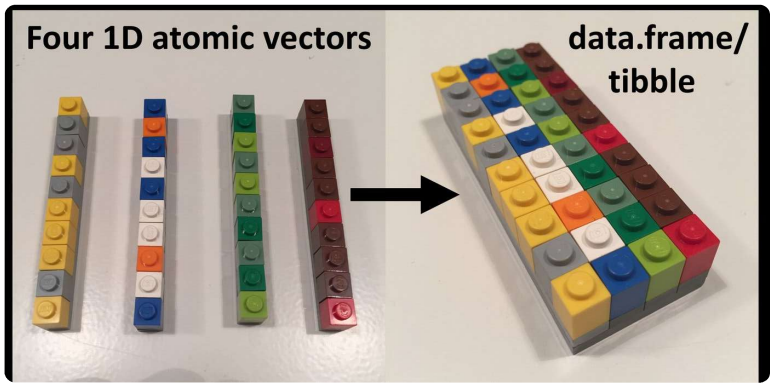
PART 1	PART 2	PART 3	PART 4
Data frames and lists	**Subsetting**	**Import**	**Export**
Columns side by side, the shape of JSON	Three kinds of brackets	CSV, Excel, and JSON	Snapshots you can share

Part 1: Data Frames and Lists

From Vectors to a Data Frame

```r
crashes_df = tibble::tibble(
  day     = names(crashes_by_day),
  crashes = crashes_by_day,
  busy    = crashes_by_day > 2000
)
crashes_df
```

```
# A tibble: 7 × 3
  day   crashes busy
  <chr>   <dbl> <lgl>
1 Mon      2020 TRUE
2 Tue      2241 TRUE
3 Wed      2327 TRUE
4 Thu      2285 TRUE
5 Fri      2380 TRUE
6 Sat      1866 FALSE
7 Sun      1664 FALSE
```



Vectors standing side by side

Each **column** is one vector with one type (`<chr>`, `<dbl>`, `<lgl>` in the header). All columns have the **same length**, so every row lines up.

The crash file is the same thing with 30,125 rows and 13 columns. `tibble` is the tidyverse's data frame: same object, tidier printing. **Part 2:** taking these apart (pick a column, pick rows).

Image: Jenny Bryan, lego-rstats.

Looking at a Data Frame

```
nrow(crashes_df)              # how many rows?
ncol(crashes_df)              # how many columns?
colnames(crashes_df)          # what are they called?
typeof(crashes_df)            # what is it, underneath?
```

```
[1] 7
[1] 3
[1] "day"     "crashes" "busy"
[1] "list"
```

```
dplyr::glimpse(crashes_df)    # one line per column
```

```
Rows: 7
Columns: 3
$ day     <chr> "Mon", "Tue", "Wed", "Thu", "Fri",…
$ crashes <dbl> 2020, 2241, 2327, 2285, 2380, 1866…
$ busy    <lgl> TRUE, TRUE, TRUE, TRUE, TRUE, FALS…
```

Four questions for any data frame

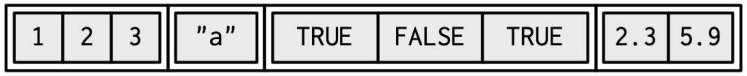
How many rows, how many columns, what are they called, and what type is each. `dplyr::glimpse()` answers all four in one call and is the first thing to run on data you did not build yourself.

`typeof(crashes_df)` says `"list"`. A data frame is a **named list of vectors** with one extra rule: every vector has the same length. Drop that rule and you get a plain list, which is next.

A List Holds Anything

```r
example_list = list(
  1:3,                    # integers 1, 2, 3
  "a",                    # one piece of text
  c(TRUE, FALSE, TRUE),   # three logicals
  c(2.3, 5.9)             # two decimals
)
str(example_list)         # the x-ray view
```

```
List of 4
 $ : int [1:3] 1 2 3
 $ : chr "a"
 $ : logi [1:3] TRUE FALSE TRUE
 $ : num [1:2] 2.3 5.9
```



Every element its own shape

A vector holds **one type, one length**. A list holds **anything**: vectors of different types and lengths, and even other lists.

`str()` prints one line per element with its type and length. Run it on anything you do not recognize; it is the fastest way to learn what an object is made of.

Image: Hadley Wickham, Advanced R, chapter 3 (Lists).

[ ] vs [[ ]] on a List

```
str(example_list[1])      # [ ]  the shaker: a list of 1
```

```
List of 1
 $ : int [1:3] 1 2 3
```

```
str(example_list[[1]])    # [[ ]] the packet: the element
```

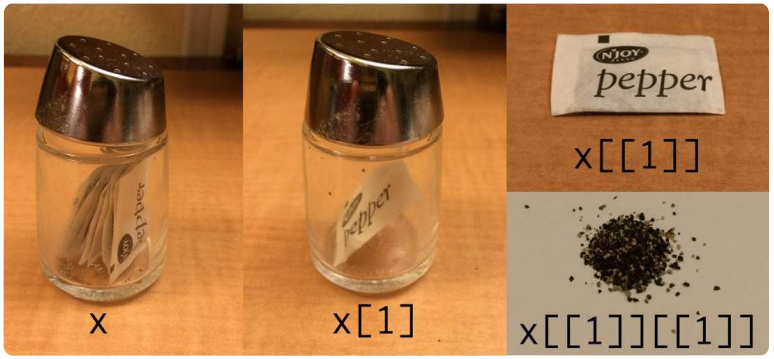
```
 int [1:3] 1 2 3
```

```
example_list[[1]][[1]]    # the pepper: first value in it
```

```
[1] 1
```

```
example_list[[4]][2]      # element 4, then its 2nd value
```

```
[1] 5.9
```



The pepper shaker rule

`x` is the shaker. `x[1]` is a shaker with one packet: **still a list**. `x[[1]]` is the packet itself. `x[[1]][[1]]` is the pepper.

When the list has **names**, `x$name` is `x[["name"]]` with less typing. Next slide.

Image: Hadley Wickham, indexing lists in R.

Examine a Job Ad

03:00

Task Answer Discussion

```r
job = list(
  title   = "Research Data Scientist, Magic Eye",
  company = "Google",
  salary  = list(min = 147000, max = 210000)
)
str(job)
```

```
List of 3
 $ title  : chr "Research Data Scientist, Magic Eye"
 $ company: chr "Google"
 $ salary :List of 2
  ..$ min: num 147000
  ..$ max: num 210000
```

Write one expression to extract each of

1. The company name.
2. The maximum salary.
3. The maximum salary again, using only `[[ ]]`.
4. The names of the elements in `job`.

Type them in a chunk in your markdown and run each line. The ad is the Google posting from Class 01.

Part 2: Subsetting and Indexing

Subsetting a Vector

```r
crashes_by_day = c(2020, 2241, 2327, 2285, 2380, 1866, 1664)
names(crashes_by_day) = c("Mon", "Tue", "Wed", "Thu",
                          "Fri", "Sat", "Sun")
crashes_by_day[2]                       # one position
```

```
 Tue
2241
```

```r
crashes_by_day["Fri"]                   # by name
```

```
 Fri
2380
```

```r
crashes_by_day[crashes_by_day > 2300]   # by condition
```

```
 Wed  Fri
2327 2380
```

Three things that can go inside [ ]

One position, counting from 1, not 0.

A name, when the vector has names.

A condition: a TRUE/FALSE vector as long as the original keeps the positions where it is TRUE.

The first two lines rebuild `crashes_by_day` so this slide stands on its own. Several positions go in with `c()`, as in `[c(1, 7)]`; a negative position such as `[-1]` drops instead of keeps.

Subsetting a Data Frame

```r
crashes_df = tibble::tibble(
  day     = names(crashes_by_day),
  crashes = crashes_by_day,
  busy    = crashes_by_day > 2000)
crashes_df$crashes                    # one column, as a vector
```

```
 Mon  Tue  Wed  Thu  Fri  Sat  Sun
2020 2241 2327 2285 2380 1866 1664
```

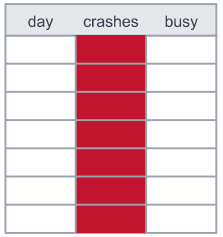
```r
crashes_df[["crashes"]]               # the same vector, by name
```

```
 Mon  Tue  Wed  Thu  Fri  Sat  Sun
2020 2241 2327 2285 2380 1866 1664
```

```r
crashes_df[1:2, c("day", "crashes")]   # rows 1 to 2, two columns
```

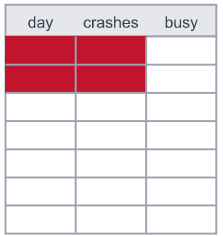
```
# A tibble: 2 × 2
  day   crashes
  <chr>   <dbl>
1 Mon      2020
2 Tue      2241
```

crashes_df$crashes



a vector (7)

crashes_df[1:2, c(...)]



a tibble (2 x 2)

`$` and `[[ ]]` open one column into a vector. `[rows, columns]` hands back a smaller tibble; leave a side of the comma empty to keep all of it.

One Table for Three Brackets

What comes back depends on the object and on the bracket. Read across a row for one object, down a column for one bracket.

Object	`[ ]`	`[[ ]]`	`$`
Vector `crashes_by_day`	a shorter **vector**	one element (rarely needed)	not allowed
List `example_list`, `job`	a shorter **list**	one element, whatever it is	one **named** element, same as `[["name"]]`
Data frame `crashes_df`	a smaller **data frame** `[rows, columns]`	one column as a **vector**	one column as a **vector**

Rule of thumb

- `[ ]` **keeps the container**: a smaller vector, list, or data frame of the same kind.
- `[[ ]]` and `$` **open the container**: you get what is inside.
- The pepper shaker again: one bracket is still a shaker, two brackets is the packet.

Predict the Output

02:30

crashes_by_day | crashes_df | job | Predict | Discussion

```r
crashes_by_day = c(2020, 2241, 2327, 2285, 2380, 1866, 1664)
names(crashes_by_day) = c("Mon", "Tue", "Wed", "Thu",
                          "Fri", "Sat", "Sun")
crashes_by_day
```

```
 Mon  Tue  Wed  Thu  Fri  Sat  Sun
2020 2241 2327 2285 2380 1866 1664
```

Part 3: Data Import

Three Commonly Used File Formats

1 CSV

Plain-text rows and commas: the workhorse of data exchange. `readr::read_csv()` reads a local file or a web address the same way.

Today: the Indeed job postings index for Ohio, served by FRED.

2 Excel

Workbooks with sheets, titles, and footnotes, formatted for people. `readxl::read_excel()` reads them from a local file.

Today: the BLS occupational projections workbook.

3 JSON

Nested name-value records, the shape of a list. `jsonlite::fromJSON()` reads a local `.json` file or one served at a web address.

Today: Cincinnati's crash reports, the source of raw data for the aggregated 2025 crash stats.

> Whatever the format, the first thing to run on what comes back, every time: **`dplyr::glimpse()`**.

Today's CSV: Ohio Job Postings on FRED

Customize | Download Data | FRED - Economic Data from the St. Louis Fed

---

- The **Indeed Job Postings Index for Ohio**: daily postings, indexed to **Feb 1, 2020 = 100**; the hiring boom of 2022, the cooling since, and the market you will graduate into.
- Behind this chart is a **CSV**: we will extract it together in class, straight from its URL, with `readr::read_csv()` and `dplyr::glimpse()` (1,827 rows: a date column and a double column).
- **Source:** Indeed Job Postings Index for Ohio (IHLIDXUSOH), FRED, Federal Reserve Bank of St. Louis.

Excel: Download First, Then Read

1 **Find the table in your browser.** BLS Employment Projections → **Fastest growing occupations** → the **Other available formats: (XLSX)** link at the top.

2 **Save it into your project.** The file is `occupation.xlsx`. In the save dialog, browse to the **top level of your repo** (next to `markdowns`, not inside it), click **New Folder**, name it `data`, and save inside it.

3 **Read it with a relative path.**
`readxl::read_excel("../data/occupation.xlsx")`, next slide: from your markdown, up one level (`..`), then into `data/`. The same line works on the lab PC, your laptop, and my machine when I grade.

Why the download this time?

`read_excel()` **cannot read a URL**: a workbook has to be a file on disk.

And BLS **turns away scripted downloads**: a browser gets the file, a script gets a 403 error page.

This is the project paying off

The file lives in `data/`, the code in `markdowns/`, both in the repo, both pushed. Whoever clones it can re-run everything with no path edits.

Reading an Excel File

```r
fastest = readxl::read_excel(
  "../data/occupation.xlsx",
  sheet = "Table 1.3",  # one of 12 sheets
  skip  = 1,            # title row above the header
  n_max = 31)           # stop before the footnotes
dplyr::glimpse(fastest)
```

```
Rows: 31
Columns: 7
$ `2025 National Employment Matrix title` <chr> "T…
$ `2025 National Employment Matrix code`  <chr> "0…
$ `Employment, 2025`                      <dbl> 17…
$ `Employment, 2035`                      <dbl> 17…
$ `Employment change, numeric, 2025-35`   <dbl> 59…
$ `Employment change, percent, 2025-35`   <dbl> 3.…
$ `Median annual wage, dollars, 2025`     <dbl> 50…
```

Paths in this course are always `"../data/SOMETHING"`: your markdown lives in `markdowns/`, your data in `data/`, so every local file you read is up one level, then into `data`.

Real workbooks are messy on purpose

`sheet`: this workbook has 12 sheets; we want Table 1.3, the fastest growing occupations.

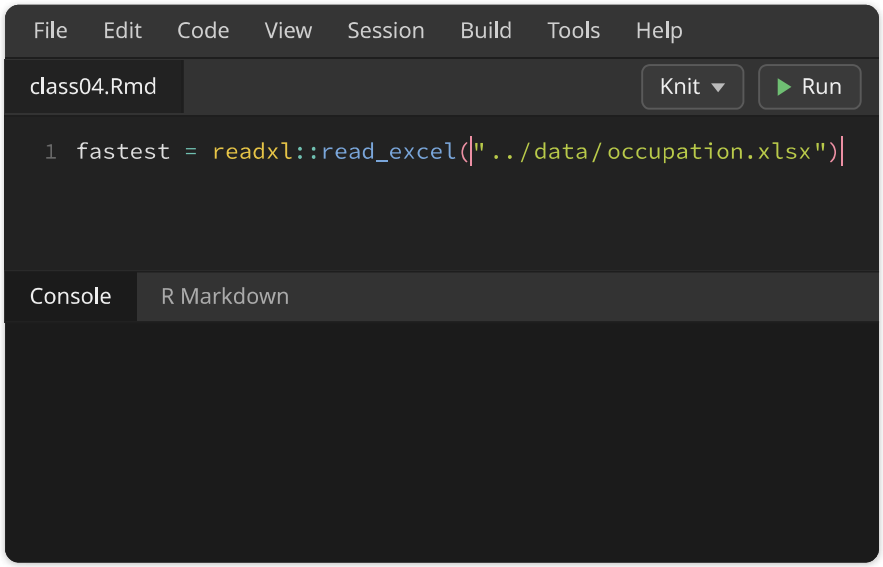
`skip = 1`: row 1 is a title for humans, not a header.

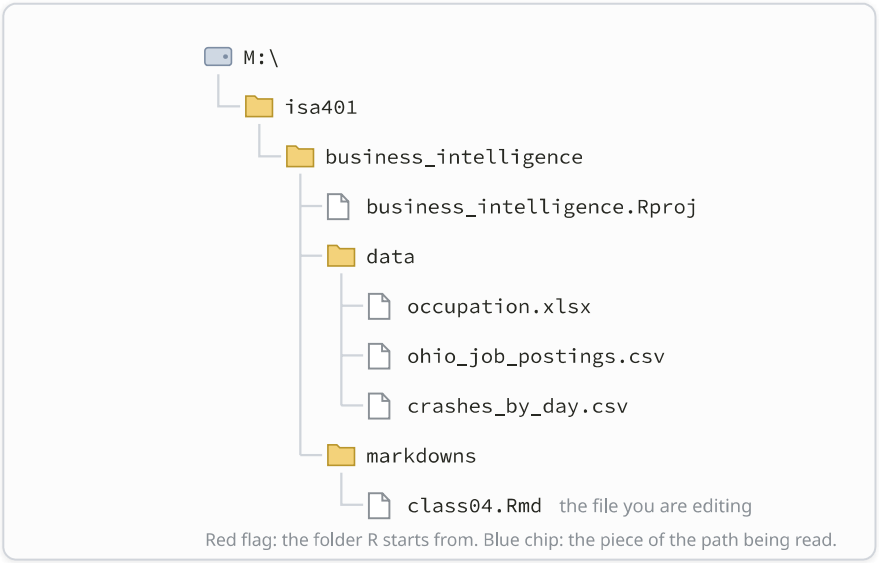
`n_max = 31`: below the data sit three footnote rows that would otherwise pollute every column.

Data scientists are the third fastest growing occupation in the country: **+34.6%** by 2035, median wage **$120,230**. You are in the right room.

Source: Table 1.3, Fastest growing occupations, 2025 and projected 2035, U.S. Bureau of Labor Statistics, Employment Projections (modified Aug 27, 2026).

Same Line, Two Starting Folders





1 Knit | 2 Run line | 3 The fix | 4 Any local file | Replay all

Play | Step | 0.5x | 1x | 2x

A Web-Served JSON File

```
json_url = paste0(
  "https://data.cincinnati-oh.gov/",
  "resource/rvmt-pkmq.json?$limit=100")
crashes_json = jsonlite::fromJSON(json_url)
dplyr::glimpse(
  crashes_json[, c("crashdate", "dayofweek",
                   "crashseverity", "age")])
```

```
Rows: 100
Columns: 4
$ crashdate     <chr> "2026-08-22T22:29:00.000", "…
$ dayofweek     <chr> "SAT", "SAT", "SAT", "SAT", …
$ crashseverity <chr> "3 - MINOR INJURY SUSPECTED"…
$ age           <chr> "20", "48", "38", NA, "27", …
```

JSON is the shape of a list

The file holds an **array of records**, like `job` from Part 1, repeated 100 times. `fromJSON()` lines the records up: names become columns, records become rows, and the result is a data frame. A `.json` file on disk reads exactly the same way.

This one is served at a web address, so it is **current the moment you run the line**: the query after the `?` asks the city for the 100 most recent reports.

Every column is character, even the dates and ages: this file quotes every value. Monday's lesson applies: what a column looks like and what R stored are different things, and `glimpse()` is how you check. Cleaning types is a week 4 job.

Source: Traffic Crash Reports (CPD), City of Cincinnati Open Data, served as JSON.

Data Import for Cybersecurity Game

06:00

The Case | Answer | Discussion

INC-2026-0902: a 02:31 VPN sign-in from abroad.

Whose account? What was taken? The logs are **files at a URL**, and you are the analyst on duty. In pairs, in a Rmd file:

1 **Identify the account.** Import the VPN sign-in log. Which account has the **most failed attempts** before a successful sign-in?

```
logins = readr::read_csv("https://isa401-import-heist.vercel.app/api/stage1")
```

2 **Scope the damage.** That account replaces **ACCOUNT** below. How many of the events in `log$events` are **DOWNLOAD**s?

```
log = jsonlite::fromJSON("https://isa401-import-heist.vercel.app/api/stage2?key=ACCOUNT")
```

3 **Close the ticket.** The count replaces **NUMBER** below. On **CLEARED**, read the report and post your team name on the **Menti wall**.

```
jsonlite::fromJSON("https://isa401-import-heist.vercel.app/api/finish?code=NUMBER")
```

Rules of engagement

1 **Open each URL in a browser tab first** and read the raw file. The evidence is small.

2 Then import it in R and `glimpse()` it. Use `[ ]`, `[[ ]]`, and `$` (Part 2) to pull the ACCOUNT out of stage 1 and the NUMBER out of stage 2.

3 Wrong key or code? **The system answers with a hint.** Read what comes back; it is data too.

The incident ticket

Briefing page: `isa401-import-heist.vercel.app`

Stage 1 is a CSV, so `read_csv()`. Stages 2 and 3 are JSON, so `fromJSON()`. Copy the calls on the left; only the **highlighted** parts change.

23 / 31

Part 4: Data Export

Writing Data Out

```
readr::write_csv(ohio_postings,
  "../data/ohio_job_postings.csv")
writexl::write_xlsx(ohio_postings,
  "../data/ohio_job_postings.xlsx")
```

Copy

Writing mirrors reading: the same `"../data/SOMETHING"` path, in the other direction.

Why export at all?

Freeze a snapshot. The web-served files answer differently tomorrow; the file in `data/` is what you saw today.

Hand it off. Most of your teammates live in Excel; `write_xlsx()` meets them there.

Watch the **Files pane** as the chunk runs: `data/` now holds the downloaded workbook plus the two files you just wrote, all inside the project, all about to be committed.

The Ritual, Now With Data

1 Knit

Click **Knit**. `class04.html` re-runs every chunk top to bottom and saves the page next to your markdown.

If the knit fails, a chunk depends on something you only ran in the Console. Fix the markdown, not the Console.

→

2 Commit

GitHub Desktop lists today's work: `markdowns/class04.Rmd`, `class04.html`, and the `data/` files.

Summary: `Add import and export markdown with data`.

→

3 Push and verify

Push origin, then check the repo on GitHub: `markdowns/` and `data/` both there, readable from any device.

That is code, data, and output, together.

If it is not pushed, it is not done. Your repo now carries code, data, and output that anyone, including future you, can re-run.

Recap

Summary of Main Points

By now, you should be able to do the following:

- **Build and inspect** a **data frame** and a **list** with `tibble()`, `list()`, `dplyr::glimpse()`, and `str()`.
- **Subset** them: `[ ]` keeps the container, `[[ ]]` and `$` open it, `[rows, columns]` on a data frame.
- **Import** a CSV straight from a **URL**, an **Excel sheet** with `sheet`, `skip`, and `n_max`, and a **web-served JSON file**.
- **Export** with `write_csv()` and `write_xlsx()` into `../data/`, then knit and push the lot.

Skills practiced: `glimpse()` on everything you import, predicting what each bracket returns, and reading a messy workbook's structure before trusting it.

Next class (Wed, Sep 9, after Labor Day): dplyr, the tidyverse toolkit that turns today's subsetting into readable pipelines.

Before You Go



Review and Clarification

- **Class Notes**: Take some time to revisit your class notes for key insights/concepts.
- **Zoom Recording**: The recording of today's class will be made available on Canvas approximately 3-4 hours after the session ends.
- **AI Recaps:** make your own with Gemini's NotebookLM, as I did for Class 01 (see the right).
- **Questions**: Do not let confusion pile up.

Make your own recap in NotebookLM

- Go to notebooklm.google.com and sign in with your Miami Google account.
- Add today's slide deck as a source: paste the deck's address from Canvas as a **Website** source.
- Click **Audio Overview** for a podcast, or **Video Overview** for a narrated video. Both take a few minutes to generate.

What it looks like: the Class 01 video and podcast. Treat them as a study aid, not a substitute for the slides: the AI summarizes what is on the page and can misstate details.

🎯 Assignments 🎯

Assignment 02: Make the Portfolio Yours

Clone your `portfolio` repo, make it an RStudio project, then edit, commit, and push until every sentence is yours. Submit the live site's address. Due date on Canvas.

Assignment 03: Git, RStudio, and Your Portfolio

A short Canvas quiz on what you did in Assignment 02. Take it after your portfolio edits are pushed. Due date on Canvas.

Before you leave: knit, then commit and push `markdowns/class04.Rmd`, `class04.html`, and your `data/` files. If it is not pushed, it is not done.

Ungraded Practice Questions

1. From `crashes_df` (**defined on slide 6**), extract the `crashes` column two different ways.

2. Get the rows of `crashes_df` where `busy` is TRUE, keeping all columns.

3. Import the national postings index: series `IHLIDXUS`. How many rows?

4. Read **Table 1.4** (most new jobs) from `occupation.xlsx` with the right arguments, then `glimpse()` it. The file is on Canvas in the Week 02 module; on that page, click **Download occupation.xlsx (404 KB)**.

5. Export `crashes_df` to `../data/crashes_by_day.csv` and find it in the Files pane.

▶ **Answers**